

Improving Autonomous Robot Localization using GPS and IMU Data Fusion: Progress Report 1

Steffen Ullmann
Florida Polytechnic University
Lakeland, FL, USA
sullmann3121@floridapoly.edu

Abstract—Accurate localization is a fundamental challenge for outdoor autonomous ground vehicles. Standalone GPS is limited by meter-level positional accuracy and a low update rate, both of which constrain precise trajectory tracking. This project investigates the fusion of GPS and IMU data on an RC-car platform to improve localization accuracy and output rate beyond what GPS alone can provide. An Extended or Unscented Kalman Filter is the primary candidate for combining measurements from a u-blox 7 GPS receiver and an MPU-6050 6-DOF IMU. The expected outcome is a fused localization system validated through outdoor field experiments, with a trajectory tracking controller as a planned follow-up application.

Index Terms—GPS, IMU, Sensor Fusion, Autonomous Robot, Localization

I. INTRODUCTION

Standalone consumer GPS receivers typically achieve 2.5–5 m circular error probable (CEP) under open-sky conditions [1], [2], with further degradation from multipath effects and brief signal outages. Their low update rate is also insufficient for real-time vehicle control. IMUs complement GPS by providing high-frequency relative motion at up to 1 kHz, while GPS corrects IMU drift over time. Fusing both yields accuracy and robustness that neither sensor achieves alone.

This project implements GPS/IMU sensor fusion on an RC-car platform (see Section V) and evaluates the resulting localization improvement. A trajectory tracking controller is planned as a secondary goal.

II. SYSTEM OVERVIEW

The platform is a 4-wheeled RC car with Ackermann steering and all-wheel drive via a BLDC motor and VESC controller. A Radxa Rock5C (RK3588S SoC) running Linux provides compute. The primary sensors are a u-blox 7 USB GPS dongle (NMEA-0183, 1 Hz) and an MPU-6050 6-DOF IMU (I2C, up to 1 kHz) [3]. The MPU-6050 carries no magnetometer, so heading relies on gyroscope integration and is subject to drift. Full specifications and photos are in Section V.

III. PROGRESS TO DATE

Sensor and actuator interfaces have been verified through standalone Python test scripts (detailed in Section VI). All code is publicly available in the project repository [4].

GPS (`test_gps_serial.py`): The u-blox 7 receiver outputs NMEA-0183 sentences at 1 Hz over `/dev/ttyACM0`. The script filters for `$GNGGA` and `$GPGGA` sentence types, which carry latitude, longitude, altitude, fix type, and the number of tracked satellites. Using `pynmea2`, these fields are parsed and printed in real time.

IMU (`test_imu_i2c.py`): Communication with the MPU-6050 was established over I2C bus `/dev/i2c-8` at address `0x68` using the `periphery` library. Writing `0x00` to register `PWR_MGMT_1` wakes the device from its default sleep state. The `WHO_AM_I` register returned `0x68`, confirming correct device identity. Temperature is read continuously from the two-byte `TEMP_H/TEMP_L` pair using the datasheet conversion formula [3]. Accelerometer and gyroscope register reads are the immediate next step.

Drive-by-wire (`uart_test.py`): A 100 Hz loop reads USB HID reports from a PS4 DualShock 4 and commands the VESC over UART. R2 controls duty cycle (0–100%), L3 horizontal controls steering servo position (0.0–1.0). A safety interlock requires pressing `OPTIONS` before accepting input. VESC telemetry (RPM, battery voltage, motor current, fault code) is parsed from the UART response stream and logged each cycle. VESC configuration is detailed in Section VII.

IV. PLANNED WORK

ROS2 integration: The test scripts will be refactored into ROS2 nodes publishing the current GPS-fix and IMU data as their own topics, enabling structured data flow and software architecture. The Rock5C’s octa-core SoC is expected to provide sufficient compute headroom for the ROS2 real-time setup.

Sensor fusion: The final algorithm will be selected based on motion-model complexity and computational budget. Candidates are the Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF), and a simpler complementary filter if full Kalman complexity proves unnecessary. Localization accuracy will be quantified by comparing GPS-only with fused position estimates.

Trajectory control: Pure pursuit or the Stanley method will be evaluated as possible candidates for Ackermann-steering trajectory tracking once fused localization is stable. Cross-track error along a predefined path will serve as the performance measure for this phase.

The 5-week project timeline is given in Section VIII.

V. APPENDIX A: HARDWARE SPECIFICATIONS

TABLE I
HARDWARE COMPONENTS OF THE EXPERIMENTAL PLATFORM.

Component	Specification
Compute	Radxa Rock5C, RK3588S (4× A76 + 4× A55), Linux
GPS	u-blox 7, USB, NMEA-0183
IMU	MPU-6050, I2C addr. 0x68, 3-axis accel + 3-axis gyro, up to 1 kHz
Motor controller	VESC (fw 7.0 / HW 410), UART 115200 baud
Steering	Servo via VESC servo output, Ackermann geometry
Manual input	PS4 DualShock 4, USB HID (/dev/hidraw0)
Frame	4-wheel RC car, all-wheel drive, per-axle differential

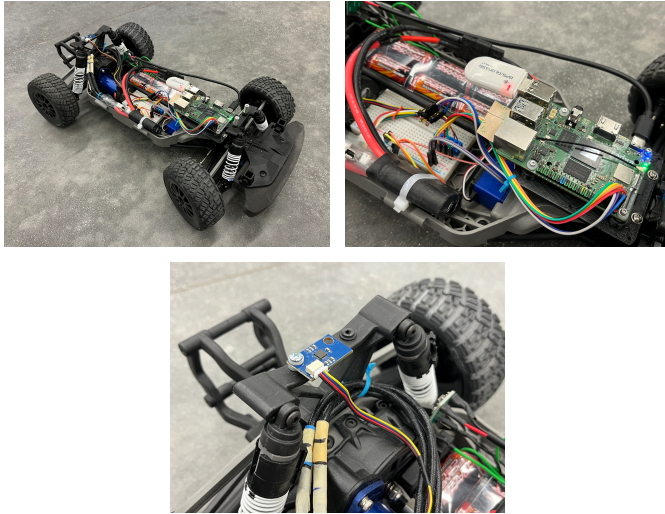


Fig. 1. Top left: full vehicle. Top right: Rock5C compute board with u-blox GPS dongle. Bottom center: MPU-6050 IMU mounted on chassis.

VI. APPENDIX B: TEST SCRIPTS

Three Python scripts in 02_Tests/ verify hardware interfaces independently before ROS2 integration.

test_gps_serial.py opens /dev/ttyACM0 at 9600 baud, filters for \$GNGGA/\$GPGGA sentences, and prints parsed latitude, longitude, and altitude using pynmea2 [5].

test_imu_i2c.py writes 0x00 to the MPU-6050 PWR_MGMT_1 register to wake the device, reads back WHO_AM_I to confirm the I2C address, and continuously polls the temperature registers (TEMP_H/TEMP_L) via the periphery library [3]. Accelerometer and gyroscope reads are the next step.

uart_test.py implements a drive-by-wire loop: reads 64-byte HID reports from /dev/hidraw0, maps R2 (byte [9]) to VESC duty cycle (0–100%) and L3 horizontal (byte [1]) to servo position (0.0–1.0), and sends SetDutyCycle and SetServoPosition commands via pyvesc at 100 Hz. VESC

telemetry (RPM, voltage, current, fault code) is parsed from the UART response stream. A safety interlock requires pressing OPTIONS before the vehicle responds to any input.

VII. APPENDIX C: VESC CONFIGURATION

The VESC motor controller is configured via VESC Tool, an open-source GUI application that communicates with the controller over USB or UART. Key parameters set during initial configuration include the motor type (BLDC), current limits, and the servo output range for the steering servo. The firmware version used is 7.0 on HW 410.

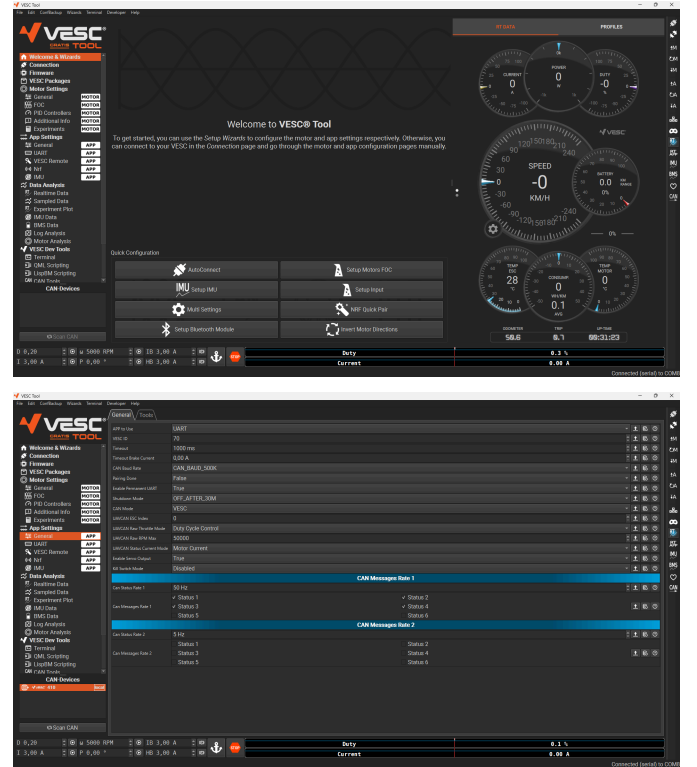


Fig. 2. Top: VESC Tool start screen showing real-time data. Bottom: General configuration settings.

VIII. APPENDIX D: PROJECT TIMELINE

TABLE II
PROJECT TIMELINE. WEEK 1 COMPLETED.

Week	Task
1	Hardware assembly, sensor interface verification ✓
2	IMU accel/gyro data reads, ROS2 environment setup on Rock5C
3	ROS2 GPS & IMU nodes, fusion algorithm selection and initial implementation
4	Fusion parameter tuning, localization accuracy evaluation
5	Trajectory controller, field testing, final report

REFERENCES

- [1] "NEO-7 u-blox 7 GPS/GNSS Modules Data Sheet." 2013. [Online]. Available: [https://content.u-blox.com/sites/default/files/products/documents/NEO-7_DataSheet_\(UBX-13003830\).pdf](https://content.u-blox.com/sites/default/files/products/documents/NEO-7_DataSheet_(UBX-13003830).pdf)
- [2] U.S. Space Force, "GPS Accuracy." [Online]. Available: <https://www.gps.gov/>
- [3] "MPU-6000 and MPU-6050 Product Specification, Revision 3.4." 2013. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Accelerometers/RM-MPU-6000A.pdf>
- [4] S. Ullmann, "GPS Autonomous Robot – Project Repository." [Online]. Available: https://github.com/SteffEng-lab/gps_autonomous_robot
- [5] Trimble Inc., "NMEA-0183 Messages: GGA – Global Positioning System Fix Data." [Online]. Available: https://receiverhelp.trimble.com/alloy-gnss/en-us/NMEA-0183messages_GGA.html